
CrocCante: CORDIC integration on the Croc SoC

VLSI 2 - ETH Zürich FS26

Francesco Maria SAPERE (ID: 25-945-999)

Luca Antonio BATTAGLIA (ID: 25-945-916)

Group: vlsi2_01fs26

1 Introduction

Digital Signal Processing (DSP) workloads and trigonometric function computation are essential and pervasive across many different fields of applications for an SoC.

Therefore, the main goal of our project is to implement a dedicated **CORDIC** (Coordinate Rotation Digital Computer) accelerator to offload the calculations of sine and cosine of a given set of angles stored in memory.

The main advantages of a dedicated piece of hardware compared to other solutions, namely software computation or static Look Up Table (LUT) are:

- **Reduced area footprint:** static memory allocation of $\sin(\theta)$ and $\cos(\theta)$ is significantly more space consuming compared to an on-the-fly computation;
- **Deterministic latency:** despite memory banking conflicts still causing minor latency fluctuations (if not handled properly), the hardware implementation is far more stable in terms of Worst Case Execution Time compared to software;
- **CPU offloading:** By offloading the heavy computations from the core, the CPU can easily either go into low-power mode (`wfi`) or execute other tasks in parallel.

1.1 Motivation

Some applications include:

- **Telecommunications:** RF SoCs need Numerically Controlled Oscillators (NCO) in digital front-ends to perform Digital Up/Down Conversion;
- **Control Systems:** Field-Oriented Control needs continuous $\sin(\theta)$ and $\cos(\theta)$ calculation for Park/Inverse-Park transforms;
- **Audio Processing & Synthesis:** audio DSP in digital synthesizers/active noise cancellation systems require high-resolution on-the-fly waveform generators.

1.2 Contribution

- **Integrated a memory-mapped CORDIC accelerator** in the user domain of the Croc SoC through an IRQ-based wrapper to handle backpressure for the computational datapath part, as well as the memory transfer/arbitration;
- **Enhanced the existing CORDIC design and explored different design solutions**, aiming at the sweet spot between throughput, latency, and area footprint;
- **Increased memory capacity** (from 4kB to 8kB) and **bandwidth** by adding two additional SRAM banks to the baseline design;
- **Increased clock frequency** from 72.46 MHz to $f_{CLK} = 75.9$ MHz (post parasitics estimation with `IHP_rcx_patterns.rules`) through architectural modifications and synthesis overconstraining;
- Explored **software solutions** for handling banking conflicts efficiently through modifications of the linker (`link.ld`).

As a result, **energy per task** has been reduced by **82.76%**, with a **numerical accuracy** of ± 10 LSB, and an **absolute speedup** of $\times 11.5$ and in terms of cycles for a workload of $N = 256$ consecutive angles.

Furthermore, given that the overhead for configuration and IRQ handling is constant with N , it is shown that the speedup increases for increasing N , which makes the current configuration appealing for potential integration in systems with more memory for workloads that require massive data batches being processed at once.

2 Architectural Improvements

The source code for the original CORDIC module can be found in the following [GitHub repository](#).

2.1 CORDIC

The **CORDIC** algorithm computes sinusoidal functions through successive vector rotations, manipulating its X and Y components using adders, shifters, and multipliers.

To guarantee compatibility with the standard cell libraries, the original CORDIC design was modified in different aspects. The active high synchronous reset was transformed into an *active low asynchronous reset* (**rst_ni**) in the whole working tree of the accelerator. Moreover, the original implementation was unrolled into a complete **16-stage pipeline**, other than pre-processing and post-processing logic. This architectural choice was crucial to guarantee timing closure at 75 MHz and reduce the critical path (Section 4.2). While we initially evaluated designs with fewer pipeline stages to reduce the area footprint, the fully unrolled architecture was ultimately retained, since its area overhead was negligible and well within our constraints.

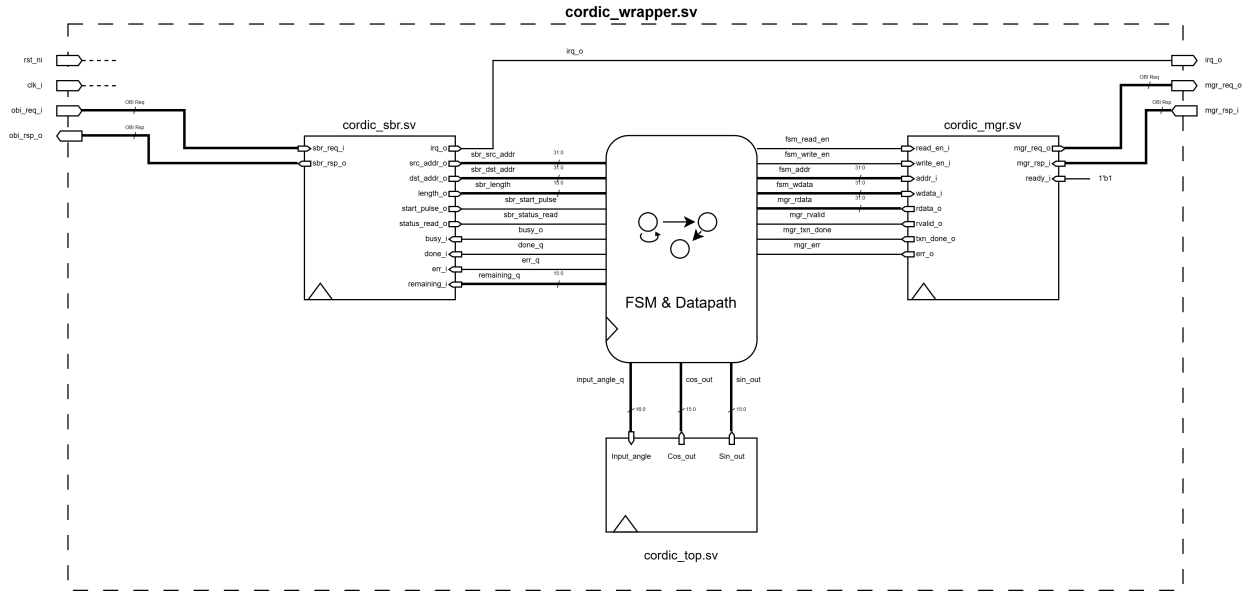


Figure 1: CORDIC wrapper architecture.

To maximize performance, we opted for a **dual interface** towards the OBI bus: a *subordinate* to receive the configuration and handle the backpressure and a *manager* that operates as a **pseudo-DMA**, relieving the CPU by reading and writing data autonomously.

2.2 CORDIC core

The CORDIC engine receives input angles from the wrapper in a 17-bit format. This input value is purposefully 1 bit wider than the internal bit size for the processing to have input angles $\phi > 2\pi$ being re-normalized before elaboration.

The input angle after normalization is then memorized in the *Z* coordinate, while *Y* is initialized to 0 and the length of the *X* component is fixed to a normalization factor $X_{in} \simeq 0.6$ to ensure that the final vector after rotations is unit-norm.

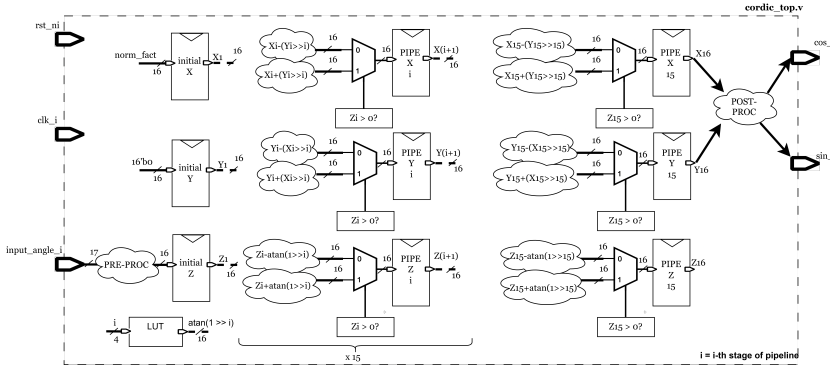


Figure 2: CORDIC core processing block diagram and iteration equations.

As discussed earlier, the math core is implemented as a **16-stage pipeline**, while the *pre-processing* reduces the angles in the standard quadrants, and the *post-processing* fixes the output signals before the output.

This brings it to a fixed latency of **19 cycles**. To track the validity of data throughout the stages, the wrapper uses a simple shift register (a valid mask that slides at every clock cycle).

The i -th pipeline stage performs a rotation of a factor 2^{-i} , as described by the iteration equations alongside the block diagram in Figure 2.

If $Z_i > 0$, a clockwise rotation happens; otherwise a counterclockwise one. The $\arctan(2^{-i})$ operation is performed by reading the value from a **LUT**.

Finally, to optimize the 32-bit OBI bus bandwidth, the sine and cosine results (fixed point 16-bit) are packed into a single 32-bit word `{sin_out, cos_out}` before being written to memory.

2.3 CORDIC wrapper

The CORDIC wrapper (Figure 1) converts the OBI transactions into CORDIC commands/data and triggers an interrupt upon completion of operations.

Offset	Register	Access	Bit(s)	Field	Description
0x00	SRC_ADDR	R/W	31:0	SRC_ADDR	Pointer to the angles array.
0x04	DST_ADDR	R/W	31:0	DST_ADDR	Pointer to the results array.
0x08	LENGTH	R/W	31:0	LENGTH	Number of elements in the batch.
0x0C	CTRL	R/W	1	IRQ_EN	Interrupt enable flag.
				START	Start execution flag.
0x10	STATUS	R-O	23:8	REMAINING	Decreasing counter for the batch.
			2	ERR	Error flag.
			1	DONE	Execution completed flag.
			0	BUSY	Execution in progress flag.

Table 1: CORDIC configuration registers mapped in the subordinate.

The wrapper state machine (Figure 3) coordinates the manager and subordinate operation. It starts from the `JOB_IDLE` state, uses the manager to read (`MGR_READ`), waits for data to flow in the pipeline (`WAIT_COMPUTE`), writes results (`MGR_WRITE`) and decreases the counter until `JOB_DONE`. The wrapper FSM was deliberately kept simple and purely sequential (processing one angle at a time) to minimize control logic complexity and ease the verification effort, since the achieved speedup ($\times 11.5$) was already vastly sufficient for the project constraints.

The wrapper design has evolved over time: in the beginning, it was thought of as a large passive subordinate, and the CPU had to pass data one by one. As we understood and measured the large offloading overhead, we opted for the pseudo-DMA fashion.

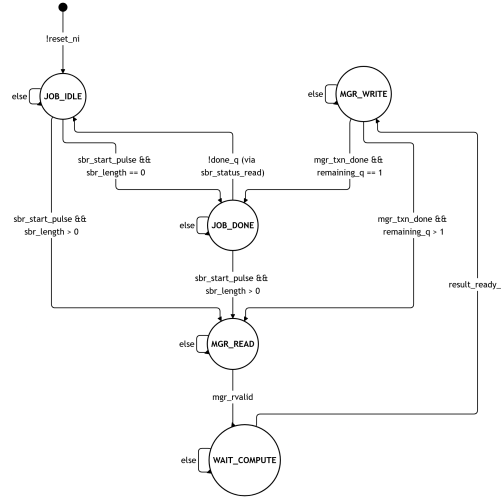
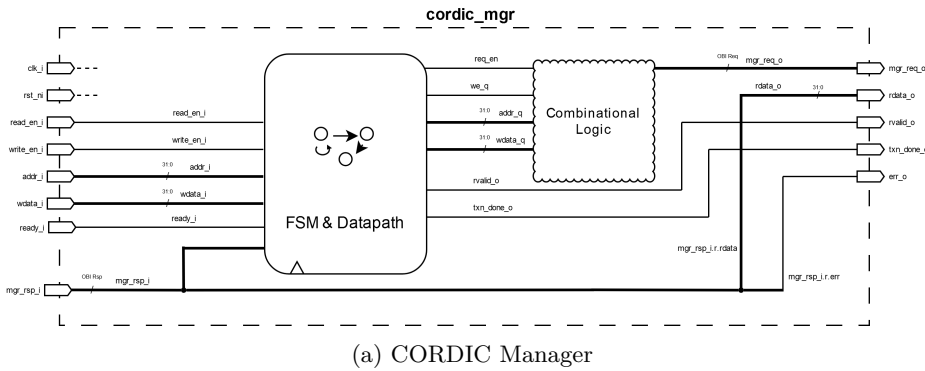


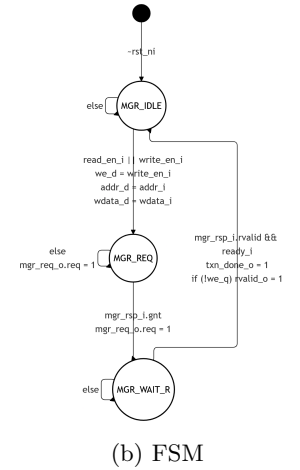
Figure 3: CORDIC wrapper state machine.

2.3.1 CORDIC manager

The **Cordic Manager** (Figure 4a) is a simple *finite state machine* with three states (MGR_IDLE, MGR_REQ, MGR_WAIT_R, detailed in Figure 4b), that converts the wrapper macro requests into OBI transactions by sending requests and addresses. Moreover, the **Cordic Manager** also automatically handles memory transfers, waiting for the valid bits from SRAM banks. This block totally hides the OBI protocol complexity from the rest of the accelerator.



(a) CORDIC Manager



(b) FSM

Figure 4: CORDIC manager block diagram and its finite state machine.

2.3.2 CORDIC subordinate

The **Cordic Subordinate** is the access port through which the CPU sends the configuration parameters for batch computing (e.g., SRC_ADDR, DST_ADDR, LENGTH, and CTRL). To protect the integrity of the ongoing calculation, the Cordic Subordinate interface implements a purely combinational backpressure mechanism: if the CPU tries to write a new configuration while the accelerator is still computing (inner state **busy** asserted), the subordinate immediately blocks the request by keeping the grant signal low. By exploiting the standard OBI handshake, this forces a safe stall on the CPU bus, stopping the transaction without any data loss until the current job is completed.

In a preliminary analysis, we have considered introducing a FIFO on the subordinate interface to allow the CPU to queue future requests, acting as a command queue. However, our final choice is a simple set of registers, as a FIFO would have been a large waste of area and power for two main reasons:

- **Bandwidth limit:** Unlike pure data streaming interfaces, the subordinate does not need to collect raw data to be computed (operation offloaded to the autonomous DMA manager), but only the 4 configuration registers. Since the OBI bandwidth physically limits the manager to a single 32-bit transaction per clock

cycle, the CPU still takes several cycles to send a complete configuration command. A deep FIFO would never have filled, but a small FIFO would have at most enqueued a single additional job.

- **Latency balancing by asynchronous offloading:** the latency penalty due to the FIFO absence is natively balanced by the offloading architecture of the system. Instead of continuously enqueueing data, the CPU programs the DMA accelerator, starts the asynchronous computation, and moves to other processes. Stalling happens only as a safety mechanism if the software tries to write too soon. Under normal operational conditions, the CPU is decoupled by the interrupt request, making the complexity of the configuration FIFO virtually useless.

2.4 SRAM banks

Leveraging the available free core area on the chip, **memory capacity** was **increased** from 4 kB (baseline) to 8 kB.

To achieve this, the trade-offs of two different approaches were considered: instantiating two larger 4 kB macros versus using four smaller 2 kB SRAMs.

Although the first option would have simplified the backend significantly, especially in terms of floorplanning and routing congestion, the four-bank topology was chosen to optimize the memory bandwidth and concurrent access capability.

The main advantages of our solution over the two-bank topology are:

- **Increased Memory Bandwidth:** Having separate banks ensures that each manager of a multi-manager system (like our implementation) can **concurrently access a different memory bank**;
- **Reduction of Banking Conflicts:** By making custom changes to the `link.ld`, software data buffers can be mapped to dedicated memory banks, reducing banking conflicts and performance penalty by ensuring that different managers operate on isolated memory regions;
- **Memory Allocation Software Flexibility:** The main limitation of having a physically partitioned memory is that the memory capacity per bank is reduced. However, having control over the software allows the programmer to allocate more memory banks to managers that perform memory-intensive tasks (e.g., NPU), while also not overallocating memory space for less memory-demanding ones (e.g., low-speed DMA channels). Although the current Croc SoC does not benefit directly from this memory architecture, we consider it to be the most scalable for further additions on the chip.

This granular address interleaving is a fundamental architectural choice to prevent OBI crossbar contention. By explicitly mapping the arrays to separate memory banks via the linker script (`link.ld`), the design enables strict parallel access: the CORDIC pseudo-DMA manager can seamlessly read or write data to Bank 1, while the CVE2 core simultaneously fetches instructions from Bank 0, effectively bypassing the crossbar arbitration bottlenecks and maintaining the `gnt` signals asserted for both managers.

The integration of the banks within the **memory address space** is shown in Table 2:

Macro / Bank	Start Address	End Address
Size: 2048 bytes (0x800) - 512 32-bit words		
SRAM Bank 0	0x1000_0000	0x1000_07FF
SRAM Bank 1	0x1000_0800	0x1000_0FFF
SRAM Bank 2	0x1000_1000	0x1000_17FF
SRAM Bank 3	0x1000_1800	0x1000_1FFF

Table 2: SRAM Memory Mapping

3 Functional Verification & SW Profiling Setup

3.1 Functional Verification

The functional verification has been divided into different phases to deeply test the CORDIC accelerator. The three main focuses were **numerical validation** of the results, *edge case* exploration (and making sure to test all the quadrants), and finally a **control logic stress test** of the backend through burst computations.

3.1.1 Vector Choice & Generation

To have a robust validation basis, we developed a Python script (`CORDIC_testvectors.py`). The script selects specific angles and calculates their real sine and cosine values using the `math` library in floating-point.

The results are multiplied by 32767 to get the best exact representation in the **Q1.15** fixed point. The obtained values are packed and formatted directly as a struct in C code that is then pasted in the software test as a golden reference for hardware. Follows a pseudo-code showing the said operations.

```
1 angle_fixed = round((angle / 360.0) * 102944)
2 sin_val = round(math.sin(angle) * 32767)
3 cos_val = round(math.cos(angle) * 32767)
4 expected_packed = ((sin_val & 0xFFFF) << 16) | (cos_val & 0xFFFF)
```

3.1.2 Accuracy

The CORDIC accelerator uses a hardware precision that is intrinsically lower than the Python floating point. The CPU reads the 32-bit output of the hardware inside `test_cordic_correctness.c`, unpacks it into two 16-bit integers (sine and cosine), and compares them with the expected result.

This architectural discrepancy arises because the reference golden model (implemented in Python) computes sine and cosine using 32-bit (or higher) floating-point arithmetic, whereas the CORDIC hardware datapath utilizes truncated 16-bit fixed-point arithmetic in the Q1.15 format. Consequently, the maximum tolerated deviation is not due to a logical imperfection in the hardware, but is rather an inevitable quantization error (rounding difference).

Since the intermediate rounding at each of the 16 hardware pipeline stages accumulates, the empirically established margin of acceptable error of ± 10 LSBs for both sine and cosine (`err_cos >= -10 && err_cos <= 10`) represents the physical mathematical limit of the Q1.15 representation for this specific algorithm. If the difference is in the $[-10, +10]$ range, the test is considered passed.

3.1.3 Edge Case Handling

The chosen vectors purposely include edge case angles on the quadrant boundaries (e.g., 1° , 89° , 91° , 179° , 269° , etc.) to verify that the *pre-* and *post-rotation* logic of the CORDIC would not introduce bugs related to the sign.

To ensure that the control logic works perfectly, we used burst tests with non-random sizes, deliberately tailored to the architecture: a quick test with **5 elements**, a test with **19 elements** (coinciding with the *pipeline latency* of the CORDIC), and one with **32 elements**, chosen as a sustained memory burst stress test, to ensure that the backend and the memory interface correctly handle continuous transactions seamlessly.

Moreover, the destination buffers were initialized with a known pattern (0xDEADBEEF). This has allowed us to check that the memory (or the DMA) overwrites exactly all the awaited data, confirming that the end of operation interrupt is not sent too early.

3.2 Performance Benchmarking Setup

The final step, after the functional verification, was the performance evaluation; that was the main aim of most of the following programs. Below is a comprehensive list of the ecosystem of C files written to profile the SoC under various aspects.

- `test_cordic_correctness.c`: The basic test evaluating algorithmic correctness and precision (Section 3.1). Furthermore, this test establishes the standard testbench interfacing used across all other tests. To enable automated simulation monitoring, `printf` logs are redirected to the memory-mapped UART and intercepted by the Verification IP (`croc_vip`), which prints them directly to the simulator console. Upon completion, the program writes the `main()` exit code to the core status register. The Verilog testbench (`tb_croc_soc.sv`) continuously polls this register over JTAG to safely halt the simulation and capture the final PASS/FAIL outcome.

- `test_cordic_HWSW_compare.c`: A benchmark to compare the execution cycles between the hardware accelerator and an equivalent C software routine (on a burst of **16 values**). This test was very useful to decompose the hardware time into precise phases: *configuration*, *execution latency*, and *interrupt handling*.
- `test_cordic_HWSW_sweep.c`: This script performs a sweep on much longer vectors ($N = 32, 64, 128, 256$). This demonstrates how the speedup of the hardware accelerator becomes more and more evident as the data size increases, since the initial setup overhead is amortized.
- `test_cordic_contention.c`: Evaluates the impact of the memory bus contention. The accelerator is set to read/write to **Bank 1**, while the CPU enters an aggressive assembly loop that keeps hammering **Bank 0** or **Bank 1**. This is needed to measure the performance dip when memory and bus are under stress.
- `test_baseline_power.c` and `test_cordic_power.c`: Dedicated C benchmarks used to stimulate the designs in the stimuli-based post-layout power simulation. These two files execute a loop with a series of CORDIC operations over a cyclically populated array. Unlike functional tests, these programs are purposely stripped of any `printf` calls, correctness checks, or UART communications within the main computation loop. This software setup avoids CPU stalls or idle periods during I/O wait, ensuring a constant and aggressive hammering of the datapath and memory bus. This maximizes the toggle rate and guarantees that the switching activity recorded in the Value Change Dump (VCD) file is representative of the peak worst-case power consumption during steady-state execution. For each design, the post-layout gate-level simulation is performed using QuestaSim, ensuring the option `-voptargs=+acc` is enabled to avoid optimization of internal nodes. The waveform dumping is enabled through `tb_croc_soc.sv`: a hardcoded delay of 1 ms skips JTAG initialization and program load, to ensure that the dumping window records actual steady-state execution for the chip. After that, the dumping window lasts 0.2 ms (to ensure VCD's size is not prohibitive), recording an activity coverage of 143911 pins annotated against the 6855 unannotated (95.45% coverage). VCD timestamps are aligned with $t = 0$ through the use of a rebasing Python script, saving the results to the final `croc_fixed.vcd`, to avoid OpenSTA alignment warnings. After this, post-layout power is computed in OpenROAD by reading the SPEF file.

4 Backend & Implementation Challenges

After the RTL integration of the CORDIC on the Croc SoC, the backend was carried out. The main challenges that emerged during this phase regarded the **floor plan** (due to the addition of the new SRAM macros), **timing-related optimizations**, and ensuring proper **routing** and **signoff readiness**.

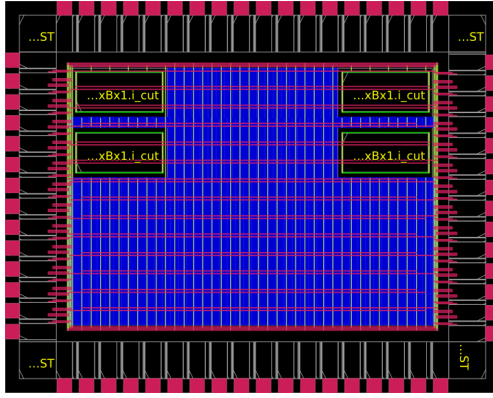
4.1 Floorplan, SRAM Placement & Power Distribution Network

The integration of the added SRAM macros during the floorplanning phase was the most challenging aspect of the backend process. Different solutions were tested and evaluated in terms of their impact on timing and routing congestion.

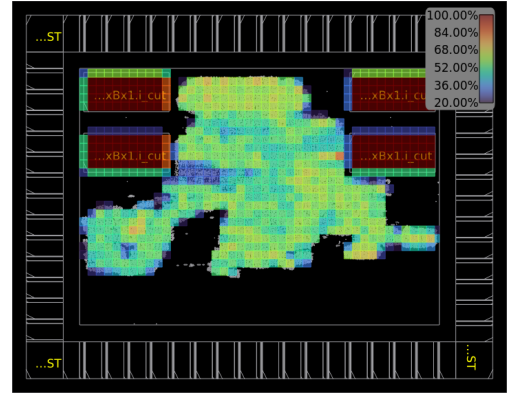
In the first configuration that we considered, the SRAMs were placed in the four corners of the core area of the chip. However, this structure was discarded because it proved to be suboptimal during the latter stages of the backend, as it forced the placement tool to position elements that frequently accessed the memory banks (e.g., the CVE2 and the CORDIC) equidistant from them, resulting in high congestion in the center area of the chip. Also, it led to poor post-SPEF timing reports due to the presence of long wires that were highly affected by parasitics.

The second solution that was considered was a single-cluster placement. This was discarded as well, since we anticipated high routing congestion and worse IR drop behavior. More specifically, in our target technology the SRAMs obstruct lower metal layers (M1 through M4), and consolidating them in a massive block would have caused severe routing issues, given that the TM1 layer over the macros is reserved for the power grid.

Therefore, the final design integrates the SRAM macros as a **double cluster topology** in the upper corners of the core area. Within a cluster, each macro is separated from the other by a margin of 105 μm to ensure sufficient space for routing. This size was chosen according to routing report analysis, aiming at having enough space to allocate for the OBI crossbar logic for each of the macros.



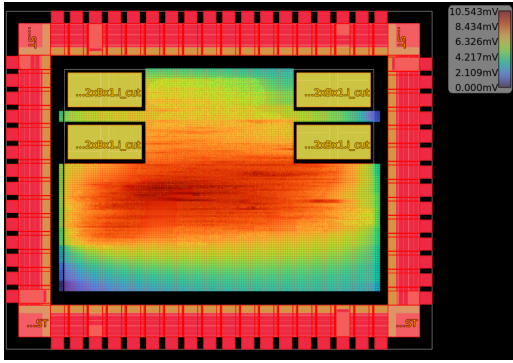
(a) Floorplan



(b) Final Density

Figure 5: Floorplan & Final Density

The IR drop analysis for both power rails shows good results for power integrity, as IR drop is kept sufficiently low for both VDD and VSS (Figure 6b).



(a) IR Drop Heat Map

Parameter	VDD	VSS
Supply voltage	1.20 V	0.00 V
Worstcase voltage	1.19 V	12.3 mV
Average voltage	1.19 V	8.73 mV
Average IR drop	7.17 mV	8.73 mV
Worstcase IR drop	10.5 mV	12.3 mV
Percentage drop	0.88 %	1.03 %

(b) Post-parasitics IR Drop @ tt Corner

Figure 6: Power Distribution Network: IR drop analysis

4.2 Timing-Related Optimizations: Synthesis, RTL & CTS

Much effort was also put into performing setup timing optimization (hold required no optimizations, resulting in 0 total violations), leveraging the available free core area. Here, different solutions were adopted during different stages of the design:

- **Synthesis:** Since the after-synthesis timing estimations are too optimistic (being agnostic to parasitics and backend development), we introduced a **mismatch** of 2.5 ns between the **target clock** in the `constraints.sdc` file and the `yosys_synthesis.tcl` script ($T_{CLK,synth} = 6.5 \text{ ns} \neq T_{CLK,sdc} = 9 \text{ ns}$). This forces the synthesis tool to upsize cells and to restructure the logic to meet this tighter timing boundary. The `.sdc` clock could have been pushed to even lower values. However, since we observed that the post-SPEF timing analysis already had a significant negative slack, we concluded that the critical path was dominated by RC delays, and that pushing it further would have resulted in unnecessarily high area occupation and routing congestion. Also, we made use of **flattening** by keeping the hierarchy (`setattr -set keep_hierarchy 1`) for the major SoC modules, as well as `cordic_wrapper.sv` and the `cordic_top.v`. The reason for this choice was letting the tool perform as many cross-boundary optimizations as possible, while still keeping the datapath of the CORDIC as a unique block, so that the different elements within it are not spread out across the core area;

- **RTL:** During the wrapper design phase, we realized that preliminary timing analyses were bounded in terms of setup by the offloading from the core to the wrapper. Therefore, we decided to:
 - Replace complicated `always_comb` processes with much cleaner FSM implementations;
 - Introduce a dedicated `obi_cut` module (Figure 7), trading off one cycle of latency with a shortened critical path.
- **Backend:** We considered increasing the number of `repair_timing` iterations during the detailed routing. However, this was not implemented in the final design, as we realized that it provided minimal improvement in spite of longer routing phases and area/congestion issues.

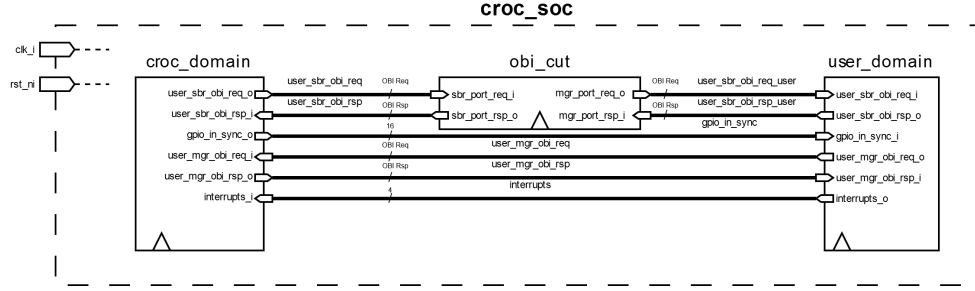


Figure 7: obi_cut RTL Integration

Critical Path against baseline & SPEF impact: The absolute Worst Negative Slack from final reports is relative to the reset generation towards one of the registers in the CORDIC wrapper. However, since the reset is regarded as a non-critical signal to define the maximum operating frequency of the implementation, this was disregarded for the characterization of the maximum achievable f_{CLK} and can be waived, considered the scope of this project.

The WNS among synchronous path changes across implementations and before/after SPEF extraction, and is summed up in Table 3.

Design Type	Startpoint	Endpoint	WNS
Baseline, post-SPEF	Instruction Register (Core)	SRAM Memory Interface	−4.80 ns
Final Design, pre-SPEF	Instruction Register (Core)	Crash Dump (Core)	−0.62 ns
Final Design, post-SPEF	Instruction Register (Core)	Register File (Core)	−4.17 ns

Table 3: WNS Across Different Implementations @ $T_{cksd} = 9$ ns, Corner: tt

We can observe that in both cases, the final design is limited by logic inside of the core, as the post-SPEF final design critical path goes through 25 combinational cells before reaching the endpoint.

Some minor tweaks could have been made to improve this path, such as using **useful skew** in **CTS** to hide this critical path with time borrowing, however this was not implemented because we already achieved a significant enough peak clock frequency improvement.

The final clock frequency after optimization is given by $f_{CLK} = \frac{1}{T_{CLK,sdc} + |WNS|}$. Therefore, the obtained results are:

$$f_{CLK,SPEF} = 75.93 \text{ MHz} \quad f_{CLK,NOSPEF} = 103.95 \text{ MHz}$$

This much of a discrepancy after parasitic extraction reveals that further improvements could have been done on a backend level, such as experimenting more with macro placement.

4.3 Detailed Routing & Signoff

To mitigate routing congestion even further, we decided to use a global cell padding by using the command `set_placement_padding -global -left 1 -right 1` during the placement phase. This prevents excessively dense packing of dense combinatorial logic, granting the detailed routing tool sufficient space to avoid M1/M2 localized DRC violations due to insufficient notch/spacing. This is especially useful for the routing channels between the SRAM macros, which was an area prone to such violations by nature.

The initial 279 antenna violations were resolved during routing thanks to the diode insertion command `repair_antennas`.

As a result, the final design is signoff ready (0 DRC and 0 LVS violations), however it still has non-zero Design Rules Violations (300 max slew, 146 max fanout, and 329 max capacitance). Although these violations may cause significant issues on a fabricated chip, they were waived for the scope of this project.

The final layout and routing congestion are shown in Figure 8

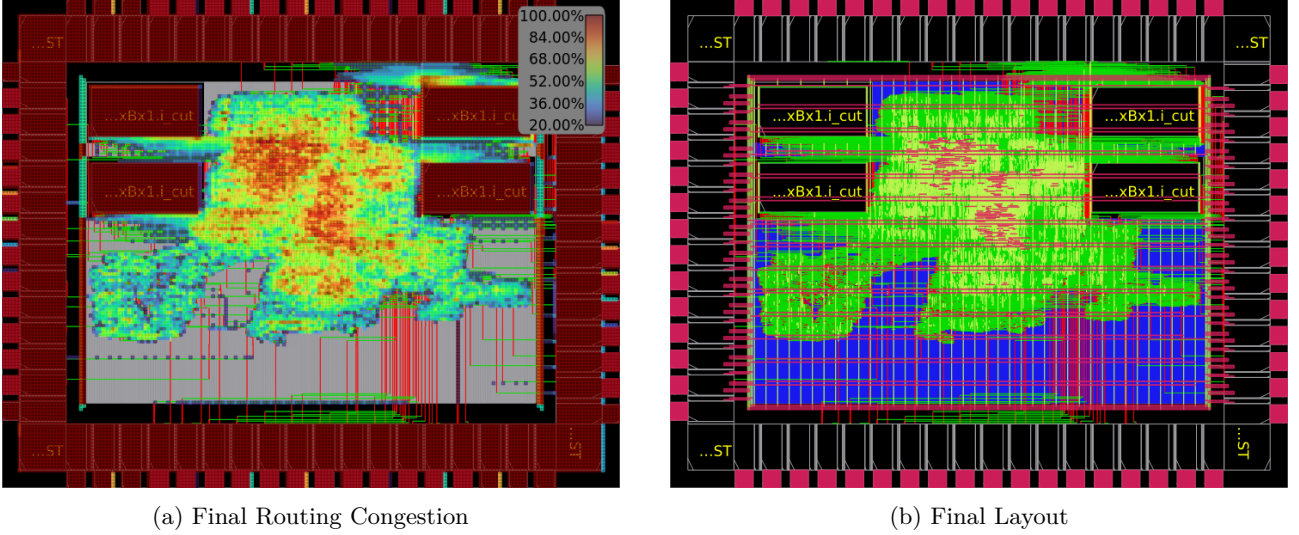


Figure 8: Final Routing Congestion & Layout

5 Performance Benchmarking Against Baseline

Using the software profiling methods described in Section 3.2, our implementation was benchmarked against the baseline in terms of **Performance**, **Power**, and **Area**. More specifically:

- **Performance:** We analyzed the **absolute speedup** in terms of **execution cycles** (to keep the analysis clock-insensitive), comparing the accelerator to the SW algorithm running on the baseline. Also, to justify the validity of the pseudo-DMA and interrupt-based approach, we detailed the execution cycles in terms of how many cycles are occupied by **IRQ overhead** and **peripheral configuration** compared against the actual **numerical operations**. Lastly, we profiled how much banking conflicts impacted the performance of our accelerator to evaluate if more clever software design was needed (i.e., modifications in the `link.ld`);
- **Power:** A **stimuli-based power analysis** using VCD (Value Change Dump) was carried out to compare power consumption for both the baseline and our design. To enable OpenSTA to correctly map the transitions, preventing timing misalignment errors, a Python script for VCD rebasing (aligning the Value Change Dump to $t = 0$) was used. Also, speedup data was used together with this profiling to define the energy savings in terms of **Energy per Task** of our implementation, showing that the power increase is compensated by the faster execution time, resulting in much lower energy costs;
- **Area:** The **area footprint** of our design was analyzed both quantitatively (in terms of kGE) and qualitatively (through OpenRoad's **Hierarchy Browser**) to highlight the spatial cost of our additions compared to the baseline.

5.1 Performance

The achieved **absolute speedup** for our accelerator, benchmarked with a sweep simulation over the value of consecutive processed angles N , shows an **asymptotic** behavior with respect to N , with a peak speedup of $\times 11.52$ for $N = 256$ (Figure 9).

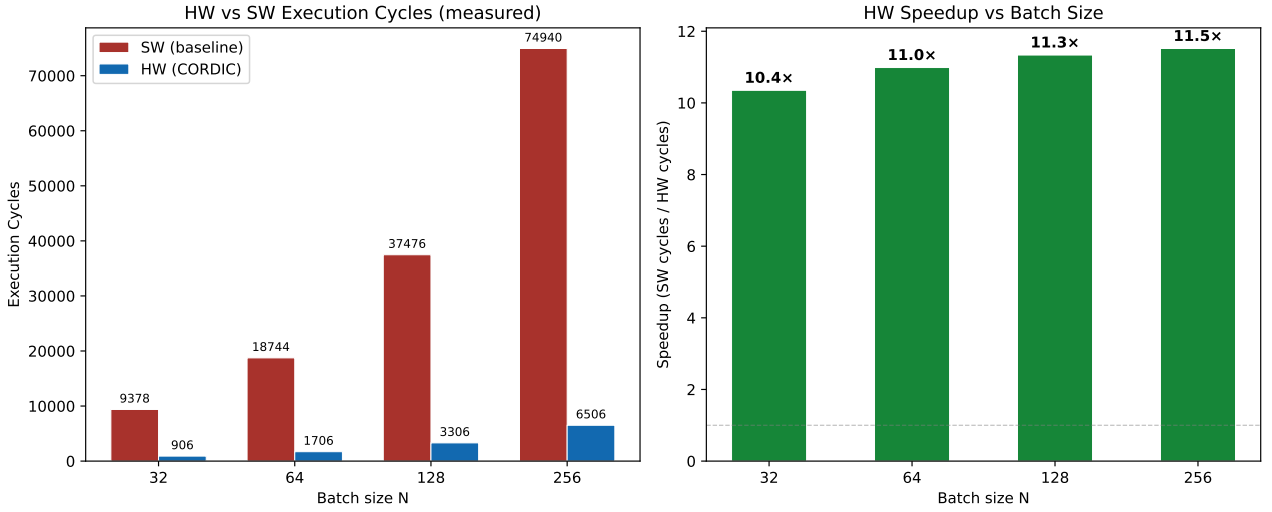


Figure 9: Speedup Scaling Against N

The **overhead** required for **configuration** and **IRQ handling** is marginal, even for small values of N (e.g., $N = 16$). Since their value is constant with the number of consecutively processed angles, this justifies the **asymptotic ideal speedup**, and therefore our accelerator vastly overperforming when compared to the SW execution, especially on large volumes of data (Figure 10).

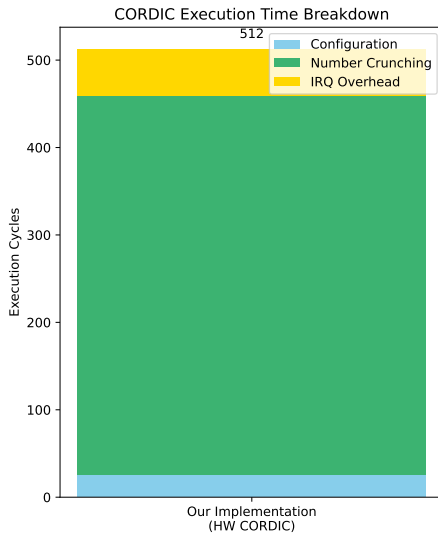


Figure 10: Execution Time Breakdown @ $N=16$

5.1.1 Banking Conflicts and link.1d Modifications

One of the main limitations of our approach to integrating the CORDIC is performance dips due to banking conflicts: since our wrapper does not include any shallow FIFOs, the execution is stalled whenever the OBI crossbar denies the `gnt` signal due to the memory being concurrently accessed by the CVE2 core. Therefore, it was decided to quantify the impact of these stalls by providing a percentage performance dip based on how many cycles are needed in case of conflicts compared to an ideal case of no other element accessing the same memory bank concurrently.

What was found is that for small values of the batch size, the stalls are marginally relevant, worsening the performance by up to 3% (Table 4).

N	$t_{\text{no conflict}}$ (cycles)	t_{conflict} (cycles)	Δ (cycles)	Performance Dip (%)
10	310	319	+9	2.9%
50	1311	1357	+46	3.5%
100	2561	2654	+93	3.6%
150	3810	3959	+149	3.9%
200	5060	5248	+188	3.7%

Table 4: Banking Conflict Impact on Performance

The performance dip in terms of absolute cycles depends both on the $\mathbb{P}[\text{conflict}]$ (which is constant with N) and the window length (the wider the window, the more cycles are wasted). Therefore, although the percentage of performance dip stays constant at around $\sim 3 - 4\%$ (as evident from Table 4), to avoid wasting more cycles (Δ) in the case of multi-manager systems ($\mathbb{P}[\text{conflict}] \uparrow$) or longer consecutive processing ($\Delta \uparrow$), it is necessary to implement software adjustments, with modifications to the `link.ld`. Since each SRAM is mapped to a fixed address window by `croc_pkg.sv`, we can split the linker memory regions so that the data is allocated on different banks explicitly for each OBI manager, as shown in Listing 1.

```

1 BANK0_START = 0x10000000
2 BANK1_START = 0x10000800 // +0x800 bytes = one 2 KiB bank
3 region CORE_RAM = [BANK0_START, 2KiB] // SRAM bank 0: core firmware
4 region CORDIC_RAM = [BANK1_START, 2KiB] // SRAM bank 1: CORDIC job buffers only
5 // CORDIC mgr uses gen_sram_bank[1], while the core uses gen_sram_bank[0]
6 place(.vectors, .text, .rodata) in CORE_RAM
7 place(.data, .bss, .stack) in CORE_RAM
8 place(.cordic_data) in CORDIC_RAM

```

Listing 1: Linker script pseudo-code configuration for SRAM memory split.

5.2 Power

VCD-based power analysis (discussed in Section 3.2) was carried out at a frequency of $f = 75$ MHz for both designs to ensure fair comparison with respect to the baseline. Its key results are summarized in Table 5.

Group	Internal (mW)	Switching (mW)	Leakage (μ W)	Total (mW)	%
Sequential	9.85	9.80×10^{-2}	1.65	9.95	34.3
Combinational	1.11	3.09	2.37	4.20	14.5
Clock	6.06	4.61	4.67×10^{-1}	10.7	36.8
Macro	3.82	0	7.92×10^{-1}	3.82	13.2
Pad	3.43×10^{-2}	3.17×10^{-1}	1.09×10^{-1}	3.51×10^{-1}	1.2
Total	20.9	8.12	5.39	29.0	100

(a) CORDIC VCD Power Report

Group	Internal (mW)	Switching (mW)	Leakage (μ W)	Total (mW)	%
Sequential	6.50	7.05×10^{-3}	1.23	6.51	44.4
Combinational	8.60×10^{-3}	8.63×10^{-2}	1.56	9.65×10^{-2}	0.7
Clock	4.05	3.05	3.51×10^{-1}	7.10	48.5
Macro	3.58×10^{-1}	0	3.96×10^{-1}	3.59×10^{-1}	2.4
Pad	5.20×10^{-2}	5.25×10^{-1}	1.09×10^{-1}	5.77×10^{-1}	3.9
Total	11.0	3.67	3.65	14.6	100

(b) BASELINE VCD Power Report

Table 5: Detailed VCD-based comparative power analysis

From the analysis of this power distribution, we concluded that:

- Most of the power consumption for both designs is related to **clock** and **sequential elements** (adding up to 71.1% for our design and 92.9% in the baseline). This suggests that potential power savings could be obtained from the implementation of **clock gating**, shutting down sequential elements and clocks whenever the accelerator is in the IDLE state;

- **Macro-related power dissipation** increased by $\times 10.64$, due to both the addition of more macros and the high utilization of the memory bus from the pseudo-DMA manager. While this figure is still acceptable for our chip (given that the Energy per Task saving is still significant, thanks to the achieved speedup), this needs to be carefully considered should power become a problem. Also, the SRAM placement on the chip during the floorplan phase should balance the tradeoff between potential thermal density issues (if they are placed too close to each other) against timing performance;
- A massive increase in **combinational power consumption** is observed ($\times 43.52$). This is due to the addition of the accelerator, which introduces a high number of parallel combinational elements (e.g., multiplexers, adders, shifters) when compared to the baseline.

Another comparative analysis of our design against the baseline in terms of power is provided in Figure 11.

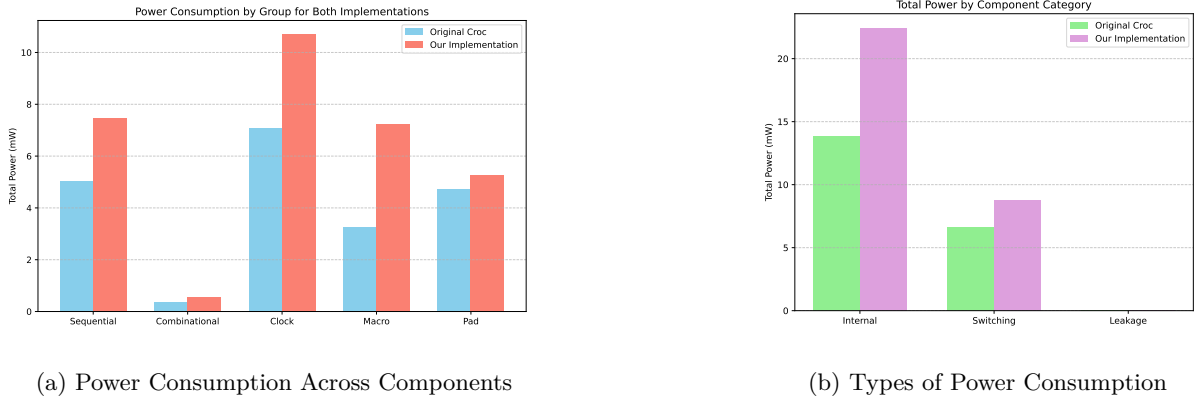


Figure 11: Power Comparative Analysis @ 75 MHz

Despite the power consumption nearly doubling (+98.63%) as a result of our additions, the **energy consumed per task** sees a significant reduction when compared to the baseline (−78.11%). This is a result of the significant speedup achieved thanks to the insertion of the accelerator. If further power-savings techniques were employed (e.g., the previously mentioned clock gating), the energy reduction could become even more significant.

Given that the absolute speedup scales with increasing values of N while power remains constant, the energy savings will get even higher for high-dimensional batches. Figure 12 shows the energy savings for a value of $N = 16$, corresponding to 78.11%.

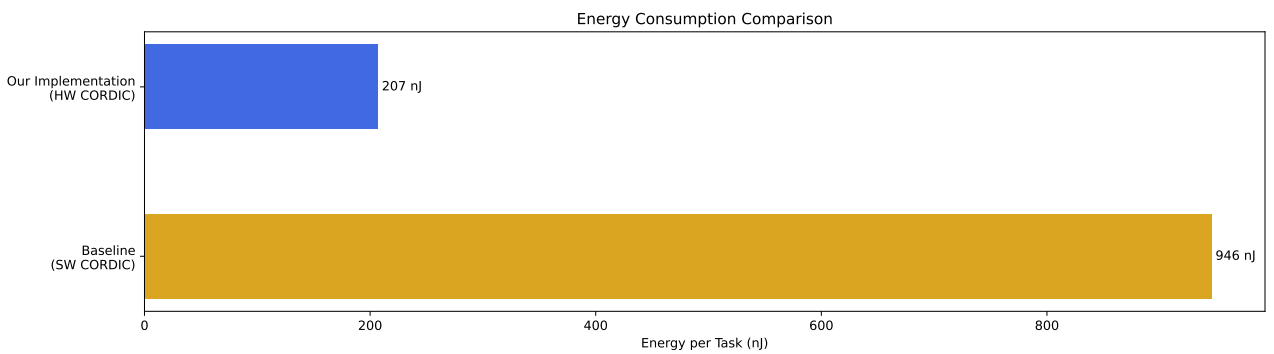


Figure 12: Energy Comparison @ $N=16$

5.3 Area

Following the addition of the CORDIC accelerator and the additional SRAM macros, a significant portion of the core area (30.8%) is still unoccupied, as shown in Figure 13.

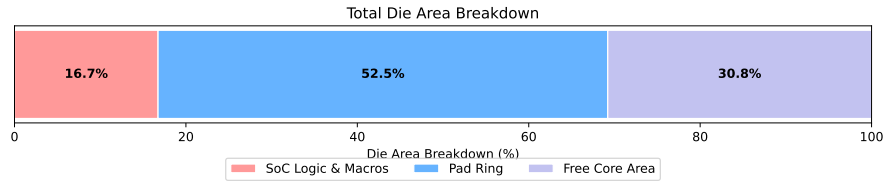


Figure 13: Area Overview

This available core area provides flexibility for possible future additions, such as additional memory or enabling optional features of the CVE2 core that are disabled by default (e.g., its fast hardware multiplier). However, for the sake of the project, we decided to not use this available area, as this led us to an easier backend process while still meeting our target improvements.

Our additions introduce a 52.57% area overhead compared to the original baseline. However, as previously mentioned, since the free core area is still significant, we avoided aggressive area optimizations for the design.

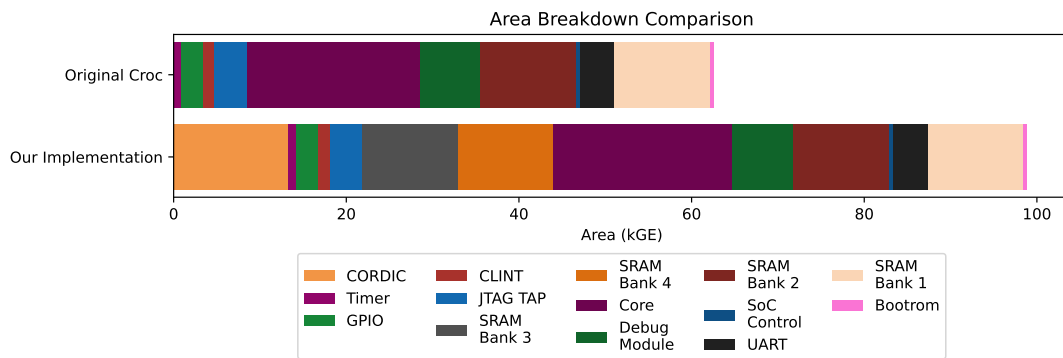
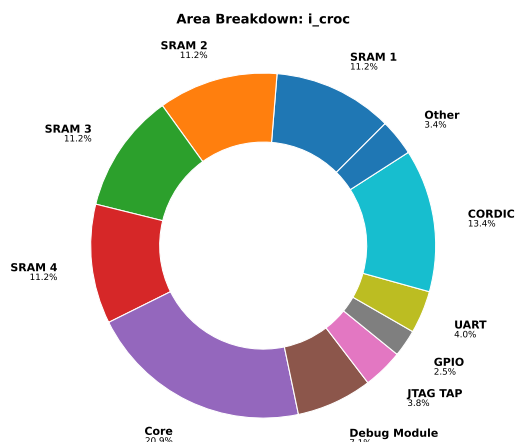


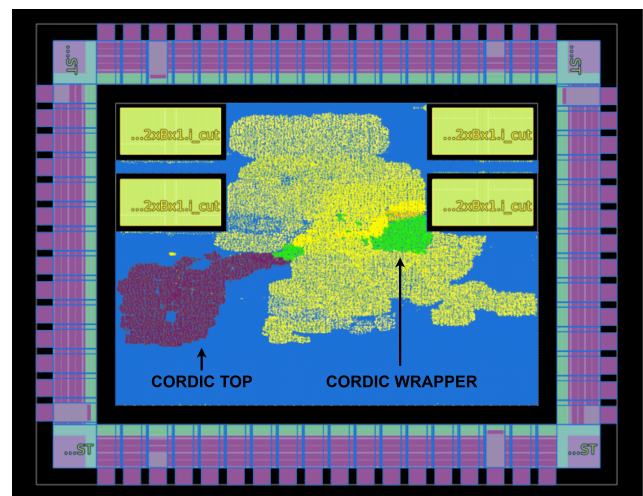
Figure 14: Area Comparison Against Baseline

Should area constraint become an issue for future developments, the most straightforward optimization would involve the tuning of the pipeline depth of the CORDIC datapath (whose area footprint is significant, as shown in Figure 15b). Since each rotation consists of a few layers of combinational logic (which are not timing-critical), it could be considered to group many rotations per pipeline stage, effectively reducing the latency and the area impact of the accelerator.

The detailed area breakdown and hierarchical visualization in OpenROAD are shown in Figures 14 and 15.



(a) Area Breakdown



(b) Hierarchical View

Figure 15: Overall Area Breakdown and Hierarchical View

5.4 Architectural Comparison Summary

The comparison of the key PPA metrics for both our design and the baseline is summarized in Table 6

Metric	Baseline	Our design	Comparison
Area (Core) (μm^2)	5.368×10^5	8.190×10^5	+52.57 %
Total Power (mW)	14.6	29.0	+98.63 %
Performance (Cycles)	74940	6506	-68434
Energy per Task (μJ)	14.58	2.51	-82.76 %

Table 6: Architectural Comparison Summary @ N=256, $f_{\text{CLK}} = 75 \text{ MHz}$

6 Conclusion and Future Work

This project successfully integrated a memory-mapped CORDIC accelerator into the Croc SoC, achieving an $\times 11.52$ execution speedup, a ± 10 LSB numerical accuracy, and a **78.11% reduction in Energy per Task**, considering a workload of $N = 256$ consecutive angles. Memory capacity was doubled, from 4 kB to 8 kB, while adopting a multi-bank structure to ensure maximum **flexibility** and **scalability** for multi-manager systems. Thanks to joint RTL-backend optimization, the post-SPEF peak operating frequency was raised from $f_{\text{CLK}} = 72.46$ MHz to $f_{\text{CLK}} = 75.93$ MHz.

Although these metrics show the effectiveness of the proposed enhancement over the baseline Croc SoC, future work could be directed to address several existing bottlenecks, such as:

- **Pipeline Depth Tuning:** The current area increase is 52.57%. A straightforward way to reduce the area footprint of the added CORDIC accelerator would be to experiment with the number of rotations per pipeline stage. Currently, each stage performs one CORDIC rotation, using a deeply pipelined datapath with 16 stages. However, since the rotations of the CORDIC were found to be non-limiting from a critical path standpoint, reducing the number of pipeline stages by employing more rotations per stage would effectively reduce latency and area occupation (which was non-critical for our design, but it could be in case of future additions);
- **Clock Gating & Power Optimization:** Power reports suggest that the clock network and sequential elements account for almost 70% of the total power consumption, resulting in a significant increase in the consumed power (98.63%). While this power increase is still justified from an energy point of view, thanks to the significant observed speedup, adding clock-gating capabilities would ensure no power is being consumed by sequential elements whenever the accelerator is in IDLE mode.
- **ISA Extension:** the datapath for the CORDIC accelerator could be integrated inside the CVE2 ALU, treating the CORDIC computation through a custom instruction and pushing the speedup near to the theoretical maximum.
- **Further Backend Design Exploration:** As discussed in Section 4, further backend runs with different configurations could mitigate some of the major bottlenecks of the existing design.